Instructions: 42 marks | 60 min | **Set 1** | 14 MCQs × 3 marks each | −1 **for wrong answer**, 0 for unattempted | Exactly ONE correct option per question | **If more than one option seems correct, pick the MOST correct one**

1. (3 points) **[MCQ]** You train an sklearn `RandomForestClassifier`, export it via `skl2onnx` to `model.onnx`, and ship it to a Swift iOS app. What is the PRIMARY advantage of shipping `.onnx` over shipping a pickled `.pkl` of the same model?

   A. `quantize_dynamic` works only on ONNX files because sklearn lacks any native quantization API
   B. Pickled models cannot be compressed by any tool, but ONNX models are auto-zipped during export
   C. Pickling the model exposes its raw training data to attackers, whereas ONNX encrypts the weights
   D. ONNX bypasses the device's CPU and forces every inference call onto the phone's mobile GPU
   E. ONNX runs in Swift / C++ / Java via ONNX Runtime; a `.pkl` file requires Python on the device
   F. ONNX silently retrains the model on the user's device to adapt to the latest incoming data drift
   G. ONNX files are human-readable JSON, making them easier to debug than the binary pickle format
   H. ONNX automatically rewrites the random forest as a neural network for faster mobile execution

   Solution

   Lecture tagline: "ONNX is the PDF of ML." Pickle is a Python-only serialization format – you cannot load it without a Python interpreter, which mobile and embedded targets do not ship. ONNX is a language-agnostic graph spec; ONNX Runtime exists for Swift, Java, JS, C++, etc. Quantization shrinks the file $\sim 4\times$ but the deployment portability is the primary reason to choose ONNX.

2. (3 points) **[MCQ]** You run an EXACT permutation test for a treatment effect on two groups of 6 subjects each (12 values total). The number of ways to re-split the 12 values into two groups of 6 is $\binom{12}{6} = 924$. The observed mean gap between the two groups is 9.0. Of the 924 re-splits, exactly 28 produce a gap as large as or larger than 9.0 (this includes the original split). What is the exact p-value?

   A. 0.030
   B. 0.020
   C. 0.012
   D. 0.970
   E. 0.060
   F. 0.500
   G. 0.090
   H. 0.045

   Solution

   The exact permutation p-value is $p = \dfrac{\#\{\text{re-splits with gap} \geq \text{observed}\}}{\#\{\text{total re-splits}\}} = \dfrac{28}{924} \approx \mathbf{0.0303}$.
   This is just a fraction. No tables, no normality assumption – the whole point of the permutation test. Distractor $0.970 = 1 - 28/924$ is what you would get if you accidentally counted the OTHER tail; 0.012 uses the wrong denominator ($28/2300$); 0.060 doubles for a two-sided alternative incorrectly.

3. (3 points) **[MCQ]** You distil a teacher model (hidden size 1024) into a smaller student (hidden size 256). To make the student imitate the teacher's HIDDEN activations, you compute the squared difference between their hidden vectors. But the teacher's vector has 1024 numbers per example and the student's has 256. They cannot be subtracted. What is the standard fix?

   A. Hidden-feature distillation is impossible; only the final output probabilities can ever be matched
   B. Add a small linear layer that maps the student's 256 numbers up to 1024, then drop it at inference
   C. MSE in PyTorch automatically broadcasts the smaller vector to the larger one when shapes mismatch
   D. Average-pool the teacher's 1024 numbers down to 256 so they line up with the student's vector
   E. Pad the student's vector with 768 extra zeros so both vectors finally have 1024 numbers
   F. Truncate the teacher's vector to its first 256 numbers, then compute the difference normally
   G. Use a 4× larger student so its hidden size grows to 1024 and matches the teacher exactly
   H. Quantize the teacher to INT8, which makes its hidden vector effectively 4× smaller in memory

Solution

Standard recipe: a small *throwaway* linear layer maps the student's 256-dim hidden up to 1024 during training, so the loss can compare it against the teacher's 1024-dim hidden. The projection is trained jointly with the student and discarded at inference – the deployed model is still the small 256-wide network. Truncation/padding/pooling all destroy directional information distillation is supposed to transfer.

4. (3 points) **[MCQ]** You prune an `nn.Linear(1024, 1024)` layer two ways:

   - **Method A:** `prune.global_unstructured(..., amount=0.5, pruning_method=L1Unstructured)`
   - **Method B:** `prune.ln_structured(..., amount=0.5, n=2, dim=0)`

   You save both and run inference on a standard CPU *without* sparse-matmul kernels. Which statement is correct?

   A. A speeds up automatically on every CPU; B requires sparse BLAS to get any speedup at all
   B. B drops whole rows so the dense matrix is genuinely smaller; A keeps the shape and still multiplies by zeros
   C. A destroys accuracy whereas B guarantees less than 1% accuracy drop on every benchmark
   D. A drops entire neurons whereas B drops individual weights chosen by smallest absolute value
   E. A is smaller on disk than B but B runs faster on a CPU once sparse kernels are enabled
   F. Both methods must first be exported to ONNX before any inference speedup can be realized
   G. A runs 2× faster on dense kernels; B runs at the same speed as the original layer
   H. Both methods reduce the file size by 50% AND double the inference speed on the CPU

Solution

Unstructured pruning sets individual weights to 0 – the matrix shape is unchanged, so a dense matmul does the same number of multiplications (just by zero). You need sparse BLAS or 2:4 hardware kernels to see a speedup. Structured pruning with `dim=0` drops whole output rows, producing a literally smaller dense matrix that any standard kernel speeds up. Tradeoff: at the same sparsity, structured loses more accuracy per weight removed.

5. (3 points) **[MCQ]** In Mixup data augmentation on a 3-class problem you sample:

   - Image A with class 0 (one-hot $y_a = [1, 0, 0]$)
   - Image B with class 1 (one-hot $y_b = [0, 1, 0]$)
   - Mixing coefficient $\lambda = 0.6$, so $x_{\text{mix}} = 0.6\, x_a + 0.4\, x_b$

   The corresponding soft target is the convex combination $y_{\text{mix}} = \lambda\, y_a + (1 - \lambda)\, y_b$. The model outputs logits $L = [\,2.0,\ 1.0,\ 0.0\,]$ on x_{mix}; after softmax this gives $p \approx [\,0.665,\ 0.245,\ 0.090\,]$, with $\ln(0.665) \approx -0.408$, $\ln(0.245) \approx -1.406$, $\ln(0.090) \approx -2.408$.

   Compute the soft-label cross-entropy loss $\mathcal{L} = -\sum_i y_{\text{mix},i}\, \ln p_i$.

   A. $\mathcal{L} \approx 0.807$
   B. $\mathcal{L} \approx 0.408$
   C. $\mathcal{L} \approx 0.245$
   D. $\mathcal{L} \approx 2.408$
   E. $\mathcal{L} \approx 0.907$
   F. $\mathcal{L} \approx 0.562$
   G. $\mathcal{L} \approx 1.814$
   H. $\mathcal{L} \approx 1.406$

Solution

Step 1 – form the target: $y_{\text{mix}} = 0.6\,[1, 0, 0] + 0.4\,[0, 1, 0] = [\,0.6,\ 0.4,\ 0\,]$.
Step 2 – soft cross-entropy: $\mathcal{L} = -\big[\,0.6{\cdot}\ln(0.665) + 0.4{\cdot}\ln(0.245) + 0{\cdot}\ln(0.090)\,\big] = -[\,0.6{\cdot}(-0.408) + 0.4{\cdot}(-1.406)\,] = -[\,-0.245 - 0.562\,] = \mathbf{0.807}$.
Common traps: 0.245 uses only the class-0 term; 0.562 uses only class-1; 1.406 ignores the soft weights altogether; $0.907 = \frac{1}{2}(0.408 + 1.406)$ uses λ=0.5 instead of 0.6.

6. (3 points) **[MCQ]** You tune the *learning rate* `lr` of a neural network using Bayesian optimization. A Gaussian-Process surrogate is fitted on 4 already-evaluated lrs and predicts, for each candidate lr, a mean μ (expected validation accuracy) and an uncertainty σ (one standard deviation). Three untried candidates:

	μ (predicted accuracy)	σ (uncertainty)
P (lr= 0.03)	0.80	0.05
Q (lr= 0.10)	0.85	0.04
R (lr= 0.30)	0.76	0.06

The acquisition function is the *Upper Confidence Bound* $\text{UCB}(\beta) = \mu + \beta\sigma$. Larger β tilts the choice toward exploration of uncertain candidates; smaller β toward exploitation of high-mean candidates. At $\beta = 2$, UCB picks Q. As you increase β, at what value does UCB FIRST switch to a different candidate, and which candidate does it switch to?

A. $\beta = 5.0$, switches to P
B. Q wins for every $\beta \geq 0$; UCB never switches
C. $\beta = 7.0$, switches to R
D. $\beta = 3.0$, switches to R
E. $\beta = 4.5$, switches to R
F. $\beta = 1.0$, switches to R
G. $\beta = 4.0$, switches to P
H. $\beta = 2.5$, switches to P

Solution

Q remains the winner as long as both $\text{UCB}(Q) \geq \text{UCB}(P)$ and $\text{UCB}(Q) \geq \text{UCB}(R)$.
P catches Q: $0.80 + 0.05\beta = 0.85 + 0.04\beta \Rightarrow \beta = 5$.
R catches Q: $0.76 + 0.06\beta = 0.85 + 0.04\beta \Rightarrow \beta = 4.5$.
R catches first, at $\beta = \mathbf{4.5}$. Check at $\beta = 4.5$: $\text{UCB}(P) = 1.025$, $\text{UCB}(Q) = 1.030$, $\text{UCB}(R) = 1.030$ (ties Q).
Any $\beta > 4.5$ flips the choice to R.

7. (3 points) **[MCQ]** You build an agent with a single tool `bash_shell(command: str)` and the standard loop:

```
messages = [user_prompt]
for _ in range(max_steps):
    response = model.generate(messages, tools=...)
    if not response.tool_calls: return response
    for call in response.tool_calls:
        messages.append({"role": "tool",
                         "content": str(execute(call))})
```

A user asks: *"Find all `.py` files and count their lines."* The model emits the tool call `bash_shell("find . -name '*.py' | xargs wc -l")`. The repo has 50,000 Python files; the shell command itself completes in 30 s. What FAILURE MODE is the loop most likely to hit on the very next iteration?

A. Verbatim shell output is appended to messages, blowing past the LLM's context window on the next call
B. The runtime auto-falls-back to a Python execution tool that is slower for this workload
C. The tool call is rejected at parse time because LLMs cannot emit raw bash strings
D. The agent enters infinite recursion because the loop has no `max_steps` guardrail
E. The runtime caches the prior successful tool result; the `wc -l` output never reaches the LLM
F. The LLM hallucinates the file paths because it has no direct access to the filesystem
G. The agent runtime rate-limits the bash tool to one shell invocation per minute on every CPU
H. The bash command output is unreadable because LLMs accept only JSON arguments and never strings

Solution

The shell command succeeds. But `wc -l` on 50,000 files emits one line per file – millions of characters – that the loop appends as a single tool-result message. The next `model.generate` sees a prompt longer than the context window and errors. Real-world guardrails: cap tool-output size before appending, summarize/truncate, or stream output to disk and feed back only a pointer.

8. (3 points) **[MCQ]** A linear layer has the weight tensor $w = [-1.20, -0.45, 0.18, 0.36, 0.93, 1.08]$. You apply symmetric INT8 quantization (signed integers in the range $[-128, +127]$). What integer is stored for the weight $w = 0.93$?

A. 112
B. 107
C. 87
D. 93
E. 127

F. 80
G. 115
H. 98

Solution

$\max(|w|) = 1.20$. Scale $s = 1.20/127 \approx 0.009449$.
$q = \text{round}(0.93/s) = \text{round}(98.42) = \mathbf{98}$.
Sanity: 0.93 is 77.5% of $\max|w|$, so $q \approx 0.775 \times 127 \approx 98$. The trap option 93 confuses the float weight with the stored integer; 127 is reserved for $\max|w|$ itself.

9. (3 points) **[MCQ]** You start a long-lived container and debug inside it:

```
$ docker run -d --name api my-app
$ docker exec -it api bash
# inside the container:
$ pip install pandas
$ echo "DEBUG=true" > /etc/myconfig
$ exit
```

Hours later you do:

```
$ docker stop api
$ docker rm api
$ docker run -d --name api my-app   # same image, fresh container
```

In the new `api` container, what is the state of `pandas` and `/etc/myconfig`?

A. Both persisted, but only on Linux hosts (macOS Docker Desktop discards them)
B. Both persisted – Docker auto-snapshots container changes back into the image on `docker stop`
C. Only `/etc/myconfig` persisted; pip-installed packages always disappear by design
D. Both persisted; Docker stores all container changes under `/tmp` on the host filesystem
E. Neither persisted – the changes lived in the container's writable layer, which is destroyed on `docker rm`
F. Neither persisted because you must use `docker stop -persist` to retain state
G. The final `docker run` fails because the name `api` is permanently reserved
H. Only `pandas` persisted; config files written outside `/app` are not retained across containers

Solution

An image is a *read-only blueprint*; a container adds a thin *writable layer* on top while it runs. Files created inside the container (`pip install pandas`, `/etc/myconfig`) live entirely in that writable layer. `docker rm` discards the layer; the new container starts fresh from the unchanged image. To persist data across container lifecycles, use a *volume* (`docker run -v ...`); to persist installed software, bake it into the image (`RUN pip install pandas` in the Dockerfile, then rebuild).

10. (3 points) **[MCQ]** You run

```
quantize_dynamic(model, {nn.Linear}, dtype=torch.qint8)
```

on a tiny MLP (two `nn.Linear` layers, sizes 64×128 and 128×10) and measure on CPU:

	Size	Latency	Accuracy
FP32 original	70 KB	0.45 ms	0.955
INT8 dynamic	22 KB	2.14 ms	0.958

INT8 is 3× smaller but ∼ 5× SLOWER. Which statement BEST explains why?

A. Dynamic quantization always runs slower than FP32, regardless of the model size or hardware
B. Quantization-aware training (QAT) is the only path; `quantize_dynamic` never improves latency
C. `nn.Linear` is excluded by default; only `nn.Conv2d` is quantized by this call
D. You need `torch.qint16` on small models to avoid an underflow that disables the kernels
E. The activation-scaling overhead per forward pass is larger than the matmul savings on small matrices
F. The accuracy went up, which means the model is in training mode and computing gradients
G. INT8 multiplications are intrinsically slower than FP32 multiplications on every x86 CPU
H. You must call `quantize_dynamic` a second time to actually replace the FP32 kernels

Solution

Dynamic quantization re-computes activation scales on every forward pass: scan input → pick scale → quantize → INT8 matmul → dequantize. For a 64×128 matmul, that overhead is larger than what INT8 arithmetic saves; large transformers amortize it and become faster. The size win (4 bytes → 1 byte) is always there; the speed win depends on matmul size and native INT8 kernels.

11. (3 points) **[MCQ]** You parse LLM-generated JSON tool outputs with the Pydantic model below:

```python
from pydantic import BaseModel, Field
from typing import List

class Review(BaseModel):
    summary: str       = Field(min_length=10, max_length=280)
    score:   float     = Field(ge=0.0, le=1.0)
    tags:    List[str] = Field(min_length=1, max_length=5)
```

Which constructor call passes `Review` validation *without raising any error*?

A. `Review(summary="ok", score=0.5, tags=["good"])`
B. `Review(summary="A clear short review.", score=0.5, tags=[])`
C. `Review(summary="A clear short review.", score=-0.1, tags=["good"])`
D. `Review(summary="A clear short review.", score=1.5, tags=["good"])`
E. `Review(summary="bad", score=0.5, tags=["good"])`
F. `Review(summary="A clear short review.", score=0.5)`
G. `Review(summary="A clear short review.", score=0.5, tags=["good"])`
H. `Review(summary="A clear short review.", score=0.5, tags=["a","b","c","d","e","f"])`

Solution

The three constraints: (i) `summary`: $10 \le$ `len` ≤ 280; (ii) `score`: $0.0 \le x \le 1.0$; (iii) `tags`: $1 \le$ `len` ≤ 5.
Failures: `summary="ok"` or `"bad"` (lengths 2, $3 < 10$); `score=1.5` or `-0.1` (out of $[0, 1]$); `tags=[]` (empty list); `tags=[...,"f"]` (6 items > 5); missing `tags` (required, no default).
Only the option with `summary="A clear short review."` (21 chars), `score=0.5`, and `tags=["good"]` (1 item) satisfies all three constraints.

12. (3 points) **[MCQ]** A team monitors drift on a single numeric feature. They collect two samples (already sorted):

Training ($n = 5$):	$10, 20, 30, 40, 50$
Production ($n = 5$):	$40, 50, 60, 70, 80$

The Kolmogorov–Smirnov statistic is $D = \max_x |F_{\text{train}}(x) - F_{\text{prod}}(x)|$ where F is the empirical CDF. Compute D.

A. 0.6
B. 0.7
C. 0.8
D. 0.5
E. 0.4
F. 1.0
G. 0.3
H. 0.2

Solution

x	10	20	30	40	50	60	70	80
$F_T(x)$	0.2	0.4	0.6	0.8	1.0	1.0	1.0	1.0
$F_P(x)$	0	0	0	0.2	0.4	0.6	0.8	1.0
$\lvert F_T - F_P \rvert$	0.2	0.4	0.6	0.6	0.6	0.4	0.2	0

$D = \mathbf{0.6}$ (reached at $x = 30, 40, 50$).

13. (3 points) **[MCQ]** Six months after a fraud-detection model went live, the team's monitoring dashboard reports:
- Recall on reviewed production cases: 0.90 → 0.55.
- Two-sample KS test on each numeric input feature: every $p > 0.30$.
- Fraud rate among reviewed cases: 0.8% then, 0.8% now.

- No release, no schema change, no preprocessing change since launch.

Which single diagnosis is MOST consistent with all four observations together?

A. Data drift
B. Training-serving skew in the feature store
C. The KS test is invalid on non-normal data
D. Label drift
E. Concept drift
F. Overfitting that only now manifests
G. Random variance; no real drift
H. Pipeline bug in preprocessing

Solution

$P(X)$ stable (KS p-values all large) rules out data drift. $P(Y)$ stable (same fraud rate) rules out label drift. Yet labelled performance collapsed – the mapping $X \to Y$ itself changed (fraudsters adopted new techniques that look benign to the old rule). That is concept drift: $P(Y \mid X)$ changed.

14. (3 points) **[MCQ]** You run uncertainty sampling on a 3-class softmax classifier. For four unlabeled samples the model outputs:

Sample	p_0	p_1	p_2
A	0.50	0.40	0.10
B	0.40	0.40	0.20
C	0.34	0.33	0.33
D	0.70	0.20	0.10

Rank by Shannon entropy $H = -\sum_i p_i \ln p_i$ – the highest-entropy sample is queried FIRST. Useful values:

$$\ln(0.10){=}-2.30,\ \ln(0.20){=}-1.61,\ \ln(0.33){\approx}-1.10,\ \ln(0.40){=}-0.92,\ \ln(0.50){=}-0.69,\ \ln(0.70){=}-0.36.$$

Which sample is queried first?

A. Sample D – the model is most CONFIDENT on it, so labeling it has highest payoff
B. Random sample – entropy is unrelated to informativeness in active learning
C. Sample A ($H \approx 0.94$)
D. Sample C ($H \approx 1.10$)
E. Sample B ($H \approx 1.06$)
F. Sample D ($H \approx 0.80$)
G. Sample A – it has the smallest gap between top-1 and top-2 probabilities
H. All four samples have equal entropy because their probabilities each sum to one

Solution

Shannon entropy is maximized for the uniform distribution; over $K = 3$ classes its max is $\ln 3 \approx 1.10$.
$H_A = 0.50(0.69) + 0.40(0.92) + 0.10(2.30) \approx 0.345 + 0.368 + 0.230 = 0.94$.
$H_B = 2 \cdot 0.40(0.92) + 0.20(1.61) \approx 0.736 + 0.322 = 1.06$.
$H_C = 3 \cdot 0.33(1.10) \approx 1.10$ (uniform $\to$ max).
$H_D = 0.70(0.36) + 0.20(1.61) + 0.10(2.30) \approx 0.252 + 0.322 + 0.230 = 0.80$.
Highest is C. Trap: "most confident" inverts the criterion.